Multimodal Product Search System

Revised report sections for Group 7 | Information Systems Analysis and Design

This revision develops the product-search feature as a coherent part of the HoodooMart e-commerce information system. It replaces the keyword-only wording in FR04 and UC02, then supplies aligned analysis, architecture, data, implementation, and acceptance-test details. Customers can search with text, speech, or a product image; all modes lead to one shared retrieval and ranking flow.

01 Requirements

Multimodal search lets a customer express the same product need through three input modes. Text search parses a typed query. Voice search transcribes speech into text and then uses the text-search pipeline. Image search encodes the uploaded image and retrieves visually similar catalogue products. The input modes differ at the start of the request; they converge before product retrieval, ranking, and display.

Input mode	Input processing	Shared search outcome
Text	Parse the query into category, explicit attributes, and preferences. Search by terms and semantic content.	Filtered and ranked published products.
Voice	Capture speech and transcribe it to text; retain the voice modality for display and diagnostics.	The same query interpretation, retrieval, and ranking used for text.
Image	Validate the upload and encode its visual content; optionally accept accompanying text.	Products ranked by image similarity, with applicable filters and preferences.

The system uses the active catalogue as its source of truth. Search is read-only: it does not change products, inventory, carts, or orders. Product detail and checkout must show or revalidate current price and availability.

1.1 Functional Requirement and Search Rules

1.1.1 FR04 Product Search and Filtering

FR04 Product search and filtering — The system shall allow visitors and customers to search published products using typed text, spoken queries, or an uploaded product image. For text and transcribed speech, it shall interpret the query and retrieve candidates using keyword and

semantic matching. For image input, it shall compare the encoded query image with catalogue product images. The system shall apply category and attribute filters, rank candidates by relevance, and allow customers to refine results using category and price controls. When the result set is too small, it shall relax eligible inferred attribute filters in a defined order, preserve the category constraint, and explain each relaxed condition to the customer.

FR04 retains the existing requirement identifier so the change remains traceable to SN01 product discovery and SN02 trustworthy information. The use case and design sections below define the acceptance evidence for each mode.

1.2 AI Services and System Constraints

1.2.1 Role of AI in Search

AI-assisted services interpret natural-language text, transcribe speech, and compare image representations to catalogue images. These services help retrieve and order candidates; the active catalogue remains authoritative for product name, price, category, and availability. A similarity score is not a product fact and does not replace product-detail or checkout validation.

1.2.2 Stakeholder Needs and Requirement Traceability

Stakeholder need SN01 (find products efficiently) and SN02 (receive trustworthy product information) lead to FR04. UC02 realizes FR04 across text, voice, and image input. TC1–TC4 verify discovery and ranking behavior, while FR02, FR05, and FR15 preserve catalogue quality and current product details.

1.2.3 Data Governance and Integration Constraints

SpeechToText and ImageEncoder are interfaces, not proof of a specific external model deployment. The implementation should record which adapter and encoder version produced an index, use compatible preprocessing for query and catalogue images, and rebuild affected vectors when an encoder or product image changes. Keep uploaded images only as long as needed for the search unless a clear search-history policy says otherwise. Let customers review or edit a transcript and explain relaxed filters.

1.2.4 Quality Criteria and Customer Control

Evaluate text and image retrieval separately with representative catalogue queries and judged relevant products. Report the catalogue snapshot, test cases, and any limitations of simulated speech or image encoders. Customers retain control through explicit category and price filters, transcript correction, and result refinement. Search output must not make purchase or stock changes.

1.3 Search Interpretation, Filters and Ranking

The query parser separates hard filters from soft preferences. A hard filter removes products that do not meet an explicit or reliably inferred constraint. A soft preference raises the position of

matching products without excluding the rest. For example, “men’s black sweater under 500,000 VND” yields category, colour, and price hard filters; “winter” in “winter coat” is a soft preference unless the customer explicitly selects it as a filter.

Rule area	System behavior
Hard filters	Category, brand, price range, colour, material, and gender can be hard filters. A parent category includes its descendants.
Soft preferences	Season, occasion, style, and warmth influence ranking but do not remove products.
Customer controls	Category and price explicitly selected in the interface take precedence over values inferred from query text.
Low-result fallback	If fewer than three candidates remain, relax inferred filters in this order: material, colour, brand, gender, then price. Never relax category. Each removed condition becomes a soft preference and is disclosed in the results.
Ranking	Rank first by text or image relevance, then by soft-preference match. Use sales and rating as tie-breakers when relevance is close.
Catalogue and stock	Show published catalogue records only. Availability is current at product detail and checked again at checkout.

1.4 Use Case UC02 Browse and Search Products

Use case element	Specification
Identifier and goal	UC02 — Browse and Search Products. Find suitable published products using text, speech, image, or category browsing.
Primary actor	Customer, including a visitor or registered user.
Supporting services	Speech-to-text service for voice input; image-encoding and search-index services for visual search.

Preconditions	The storefront is available and the catalogue contains published products with usable product images and category data.
Trigger	The customer submits text, speaks into the search control, uploads an image, or opens a category.
Main success scenario	(1) The system presents the search field with microphone and image-upload controls. (2) The customer chooses an input mode and submits a query. (3) Voice is transcribed; text is parsed; or an image is encoded. (4) The system creates a normalized query representation. (5) It applies customer-selected and query-derived filters, retrieves keyword and/or visual-semantic candidates, and ranks results. (6) The system displays products, interpreted filters, and any relaxed conditions. (7) The customer can refine results or open a product detail.
Alternative A — No match	The system explains that no products match and offers category browsing or broader filters.
Alternative B — Few matches	The system relaxes eligible inferred filters in the defined order, preserves category, and tells the customer which conditions were relaxed.
Alternative C — Voice unavailable	The system reports a transcription problem and lets the customer retry or edit the recognized text.
Alternative D — Invalid image	The system rejects unsupported or unreadable files with an actionable message and preserves any accompanying text.
Alternative E — Search service unavailable	The system reports the issue and offers retry or category browsing.
Postconditions	Search history may record the input mode, normalized query, filters, and displayed result order for service evaluation. Product, inventory, and order records remain unchanged.
Business rules	Search results contain published products only. Search results and details use the same active catalogue record. Checkout revalidates stock and price.

1.5 Overall Use Case Diagram and Search Module

The overall HoodooMart use-case diagram includes Customer, Administrator, Warehouse Staff, Manager, Shipping Provider, Payment Provider, Supplier, and Sales Staff. Customer use cases include Manage Shopping Cart, Manage Account, Place Order, Manage My Orders, and Browse and Search Products. Image Search, Keyword Search, and Voice Search specialize product browsing and search. The wider model also includes catalogue, promotion, inventory, supply, delivery, fulfilment, payment, customer-service, reporting, and review functions. Figure 1 shows this system context; UC02 remains focused on product discovery.

02 Analysis

2.1 Search Activity Flow and Business Context

Stage	Text	Voice	Image
1. Capture	Type query	Record speech	Upload image; optional text
2. Normalize	Parse query	Transcribe, then parse	Encode visual content; parse optional text
3. Build query	Create QueryRepresentation with modality, text, filters, preferences	Same representation with voice modality and transcript	Same representation with image vector and any text constraints
4. Retrieve	Parallel keyword and semantic top-N candidates	Same as text	Top-N visual matches from precomputed image vectors; optional text candidates
5. Filter and rank	Apply customer-selected filters early; re-rank top-K by relevance, preferences, and tie-breakers	Same as text	Apply customer-selected filters early; re-rank top-K by visual relevance and preferences
6. Present	Show products, active filters, and relaxation notices	Show products and recognized query	Show visually similar products and active filters

This flow targets relevance and latency: text and voice use parallel keyword and semantic search, while image input uses a precomputed image-vector index. Customer-selected filters are applied before re-ranking a bounded top-K candidate set. Measure result relevance at top-K and p95 response time to check the trade-off.

2.1.1 Mermaid Activity Flow (Source)

```
flowchart TD

    Start((Start)) --> Mode{Choose input mode}



    Mode -->|Text| TextInput[Enter keywords]

    TextInput --> TextQuery[Normalize text query]



    Mode -->|Voice| VoiceInput[Record voice]

    VoiceInput --> Transcript[Transcribe and normalize text]



    Mode -->|Image| ImageInput[Upload product image]

    ImageInput --> ImageVector[Encode image vector]



    TextQuery --> Query[Build modality-aware query]

    Transcript --> Query

    ImageVector --> Query



    Query --> Filters[Apply explicit filters]
```

```
Filters --> Retrieve[Retrieve top-N from the relevant index]

Retrieve --> Rerank[Re-rank top-K candidates]

Rerank --> Display[Display ranked results]

Display --> End((End))
```

2.1.2 Order and Payment Activity Diagram

This activity diagram provides the business context for product search: a customer selects a product, the system checks availability, and an available product is shown in the cart. The customer can return to the cart or confirm order creation. After an order is created, the customer chooses cash on delivery (COD), an e-wallet, or a bank card. COD is recorded; e-wallet and card payments are authorized or validated. Approved digital payments place funds in escrow before delivery; a declined payment displays an error. The diagram does not show a retry route after that error. Product search supports discovery before this flow, while cart, order, payment, and delivery belong to the wider HoodooMart system. (See Figure 2.)

2.2 Analysis Class Model and Domain Entities

2.2.1 Entity Extraction and Class Responsibilities

Class or interface	Responsibility
SearchUI	Accept typed queries; display interpreted filters, results, and messages.
VoiceInput	Capture speech and send it for transcription.
ImageUpload	Accept a supported image and optional text.

SearchController	Receive view actions and forward them to the application layer.
SearchOrchestrator	Coordinate input normalization, retrieval, ranking, and response assembly.
SpeechToText	Convert speech to transcript behind a replaceable interface.
QueryService / QueryParser	Build a normalized QueryRepresentation with modality, filters, preferences, and text.
ImageEncoder	Convert uploaded image content into a representation usable by the visual index.
SearchService	Apply hard filters and retrieve candidate products from keyword and vector indexes.
RankingService	Combine relevance, soft preferences, sales, and rating into an ordered result list.
ProductRepository	Read active product records and resolve category descendants and product details.
QueryRepresentation / SearchResult	Represent one search request and its ranked product results; SearchResult links a query to a Product and stores rank.

QueryRepresentation is shared across modes so downstream components do not need separate text, voice, and image search pipelines. It records the modality and only the representations relevant to that request. SearchResult links a query to a product because the same product can appear in many searches and at different ranks. The domain entity model in Figure 3 shows Product, Category, ProductImage, SearchQuery, SearchFilters, QueryMedia, SearchResult, Customer, Cart, CartItem, Order, OrderItem, Payment, and Review. Its SearchModality values are KEYWORD, VOICE, and IMAGE. The use-case diagram calls typed input “Keyword Search,” while the sequence diagram labels that branch TEXT; these names refer to the same typed-query path. The model links searches to filters, optional media, and ranked products; customers to searches, carts, orders, and reviews; and products to categories, images, cart/order items, results, and reviews.

2.3 Analysis Sequence Diagram

Figure 4 records the search request sequence across Customer, SearchPage, SearchController, SearchOrchestrator, SearchSubsystem, and SearchResultView. SearchSubsystem interprets text, transcribes voice, or encodes an image with optional text,

then retrieves and ranks candidates. When fewer than three candidates remain and a filter can be relaxed, the subsystem relaxes the next filter, returns a notice, and searches again. The controller returns a SearchResponse; the result view displays products, no-results feedback, or an error for invalid input or an unavailable service.

03 Design

3.1 Layered and Package Architecture

The system follows the three-layer architecture shown in Figure 5: Presentation; Application and Intelligence; and Data and Infrastructure, with external systems beyond the application boundary. The presentation layer includes product detail, cart, search, voice-input, image-upload, and result-view components with their controllers. The application layer connects CartService, InventoryService, OrderService, PaymentService, and SearchOrchestrator with search, ranking, query, image-encoding, and speech-to-text services. The data layer provides cart, order, product, and payment repositories, product and order databases, keyword and vector indexes, and image storage. Payment Gateway, Shipping Provider, Image Embedding Model, and Speech Recognition Model are external integrations. The table below narrows this architecture to the search module; Figure 5 shows how it fits into the wider cart, order, and payment workflows.

Layer	Main components	Responsibility and dependencies
Presentation	SearchPage, VoiceInput, ImageUpload, SearchResultView, SearchController	Accepts text, voice, and image input; validates basic request data; invokes the application layer; and displays products, interpreted filters, transcripts, relaxation notices, and error messages.
Application and Intelligence	SearchOrchestrator, QueryService, SearchService, RankingService, SpeechToText, ImageEncoder	Coordinates the multimodal search use case, converts different input modes into a shared query representation, applies filtering and relaxation rules, retrieves candidates, and ranks the results.
Data and Infrastructure	ProductRepository, KeywordIndex, VectorIndex, ProductDatabase, ImageStorage	Provides published product data, keyword lookup, vector similarity search, image references, and persistence operations required by the application layer.

The principal dependency direction flows strictly from top to bottom:

Presentation → Application and Intelligence → Data and Infrastructure

`SearchController` serves as the primary entry point from the Presentation Layer, forwarding a `SearchRequest` to `SearchOrchestrator` without directly accessing lower-level repositories or databases.

Key domain components—such as `SpeechToText`, `ImageEncoder`, `ProductRepository`, `KeywordIndex`, and `VectorIndex`—are defined as interfaces. Concrete infrastructure adapters implement these interfaces, enabling AI models and storage providers to be updated independently.

External AI platforms (e.g., Speech Recognition and Image Embedding services) reside outside the core application boundaries and are accessed exclusively through infrastructure adapters.

3.1.1 Package Structure

`presentation`: SearchPage, VoiceInput, ImageUpload, SearchResultView, SearchController

`application`: SearchOrchestrator, QueryService, SearchService, RankingService

`application.ports`: SpeechToText, ImageEncoder, ProductRepository, KeywordIndex, VectorIndex

`domain`: Product, Category, QueryRepresentation, SearchResult, SearchResponse

`infrastructure`: ProductDatabase, ImageStorage, Repository/Index/AI Adapters

Separating search retrieval and ranking responsibilities ensures indexing strategies can evolve without altering presentation layer elements.

3.2 Design Class Diagram

Figure 6 shows the design classes that execute multimodal search. SearchPage is the boundary for typed, spoken, and image input; SearchResultView presents products, transcripts, filters, relaxation notices, and errors. SearchController delegates to SearchOrchestrator. The orchestrator coordinates QueryService, SpeechToText, ImageEncoder, SearchService, and RankingService. SearchService uses ProductRepository, KeywordIndex, and VectorIndex; these are interfaces so concrete storage and model adapters can change independently.

`SearchPage` and `SearchResultView` act as boundary classes. User actions trigger requests in `SearchController`, which delegates execution to `SearchOrchestrator`.

The core collaborators managed by `SearchOrchestrator` include:

- **`QueryService`**: Creates a unified `QueryRepresentation` across text, voice, or image modes.
- **`SpeechToText`**: Transcribes audio streams into text strings.
- **`ImageEncoder`**: Validates image payloads and generates feature embeddings.
- **`SearchService`**: Retrieves candidate sets and enforces hard filtering.
- **`RankingService`**: Computes composite relevance and sorts candidate items.
- **`ProductRepository`**: Accesses authoritative catalogue data.
- **`KeywordIndex`** / **`VectorIndex`**: Perform specialized lexical and vector similarity lookups.

All input modes produce a shared `QueryRepresentation` object containing normalized text, input modality, hard filters, soft preferences, and vector references.

Modality components are specified via interfaces to guarantee loose coupling. Downstream transactional processes (carts, checkout, stock updates) remain isolated in distinct domain models.

3.3 Design Sequence Diagram — Search Request

The sequence diagram details the end-to-end execution flow during a search request:

1. The customer submits search criteria from `SearchPage` through `SearchController` to `SearchOrchestrator.execute(request)`.
2. Modality processing branches based on input type:
    - **Text**: Calls `QueryService.fromText(text, filters)`.
    - **Voice**: Invokes `SpeechToText.transcribe(audio)`, then `QueryService.fromTranscript()`.
    - **Image**: Runs `ImageEncoder.validate()` and `encode()`, then `QueryService.fromImage()`.
3. Candidate retrieval and fallback rules are executed by `SearchService`:
    - Queries `ProductRepository` for hard filters and queries indexes (Keyword/Vector).
    - Merges candidate sets and deduplicates records.
    - If results are under threshold (<3 items), relaxes inferred filters progressively while preserving category and customer-selected constraints.
4. Scoring and ranking are performed by `RankingService` using query relevance, soft preferences, sales volume, and rating tie-breakers.
5. A `SearchResponse` is assembled with ranked items, active/relaxed filters, and metadata, then returned to `SearchResultView`.

3.4 Entity Classes, Database, and Physical ERD

Figure 7 shows the physical tables for search, catalogue, cart, order, payment, and review data. It separates the authoritative product catalogue from query logs and ranked search results, while retaining foreign-key links to products and customer transactions.

3.4.1 Catalogue Entities

`Product` serves as the core authority for names, descriptions, attributes (category, brand, colour, material, gender), pricing, stock levels, sales metrics, and publication status.

`Figure 3 gives Category the attributes categoryId, name, and description; it does not show a parent-category attribute. Figure 7 has a category_id column on product but does not include a category table. Hierarchical category matching therefore needs a parent key and category table added to the physical model before implementation.`

`ProductImage` stores media paths, primary indicators, encoder versions, and external vector index references.

3.4.2 Customer and Transactional Entities

Figure 3 includes Customer, Cart, CartItem, Order, OrderItem, Payment, and Review. Figure 7 maps the transactional tables to cart, cart_item, customer_order, order_item, payment, and review; OrderItem stores unit_price and product_name_snapshot for purchase history. The physical diagram does not include a customer table even though cart, customer_order, and review contain customer_id, so the Customer foreign keys need a corresponding table or an explicit external identity reference.

3.4.3 Search and Log Entities

Application-level `QueryRepresentation` instances map into normalized relational log tables:

- **`search_query`**: Stores modality, raw/normalized query text, customer ID, and timestamps.
- **`search_filter`**: Logs individual interpreted filters, relaxation flags, and explicit/inferred status.
- **`query_media`**: References uploaded temporary audio or image files.
- **`search_result: Associates each query with displayed products and records rank_position, relevance_score, preference_score, final_score, an optional explanation, and created_at without duplicating mutable product data.`**

3.4.4 Search Indexes

Lexical keyword indexes and vector indexes are derived structures rather than authoritative tables. ProductImage stores image and vector references plus encoder_version; query_media stores temporary media references and expiry. Unpublishing a product removes it from active search indexes. The physical model also maps Order to the customer_order table and keeps OrderItem product-name and unit-price snapshots for purchase history.

04 Implementation and Testing

4.1 Code Structure and Integration

The implementation should depend on interfaces rather than provider-specific calls. SpeechToText may use a simulated adapter for a classroom demo and a speech service for deployment. ImageEncoder may use a lightweight demo encoder or a compatible image-text embedding model. KeywordIndex and VectorIndex may be replaced independently. The report

should distinguish a simulated adapter used in the demo from a production service; an interface alone does not mean an external provider has been integrated.

For image retrieval, catalogue product images must be encoded with the same compatible encoder and preprocessing used for query images. The application should validate file type and size, handle missing product images, and avoid retaining uploaded images longer than required by the search and any disclosed search-history policy.

4.2 Black-Box Testing and Demo

Test	Input and expected behavior	Evidence
TC1 Text search	“Men’s black sweater under 500,000 VND” yields category, colour, and price constraints; relevant sweaters rank first.	Show interpreted filters, result count, and the first three products.
TC2 Voice search	“I want a winter coat” is transcribed; coat category is applied and winter is treated as a soft preference.	Show transcript, modality, and preference-based ordering.
TC3 Image search	Upload a navy sweater image; visually similar published products rank first.	Show uploaded image and top results; verify the image encoder and index use compatible representations.
TC4 Filter relaxation	A query with fewer than three matches relaxes eligible conditions in order, never drops category, and reports each relaxed filter.	Check resulting filter chips, explanation, and ranking.
TC5 Read-only behavior	Search and result refinement do not modify catalogue, stock, cart, or order records.	Compare relevant records before and after the search.

4.3 Evaluation and Acceptance Criteria

Evaluate retrieval quality separately for text, voice, and image using representative queries, a stated catalogue snapshot, and judged relevant products. Report response time under a defined workload and document limits of simulated speech or image services.

Negative and integration checks should cover failed transcription, unsupported image type, missing image vectors, and unavailable indexes. For each case, record the user-visible message, system fallback, and resulting state; confirm that no catalogue, inventory, cart, or order record changes.

4.4 Conclusion and Traceability

FR04 and SN01 map to the shared text, voice, and image search flow verified by TC1–TC4. SN02, privacy, FR02, and FR15 map to published catalogue data, current product details, upload validation, and bounded image retention, checked through catalogue-consistency and upload-error tests. FR05 maps to the current ProductRepository record and checkout revalidation.

Appendix A. System Diagrams

Figure 1. Overall HoodooMart use-case diagram

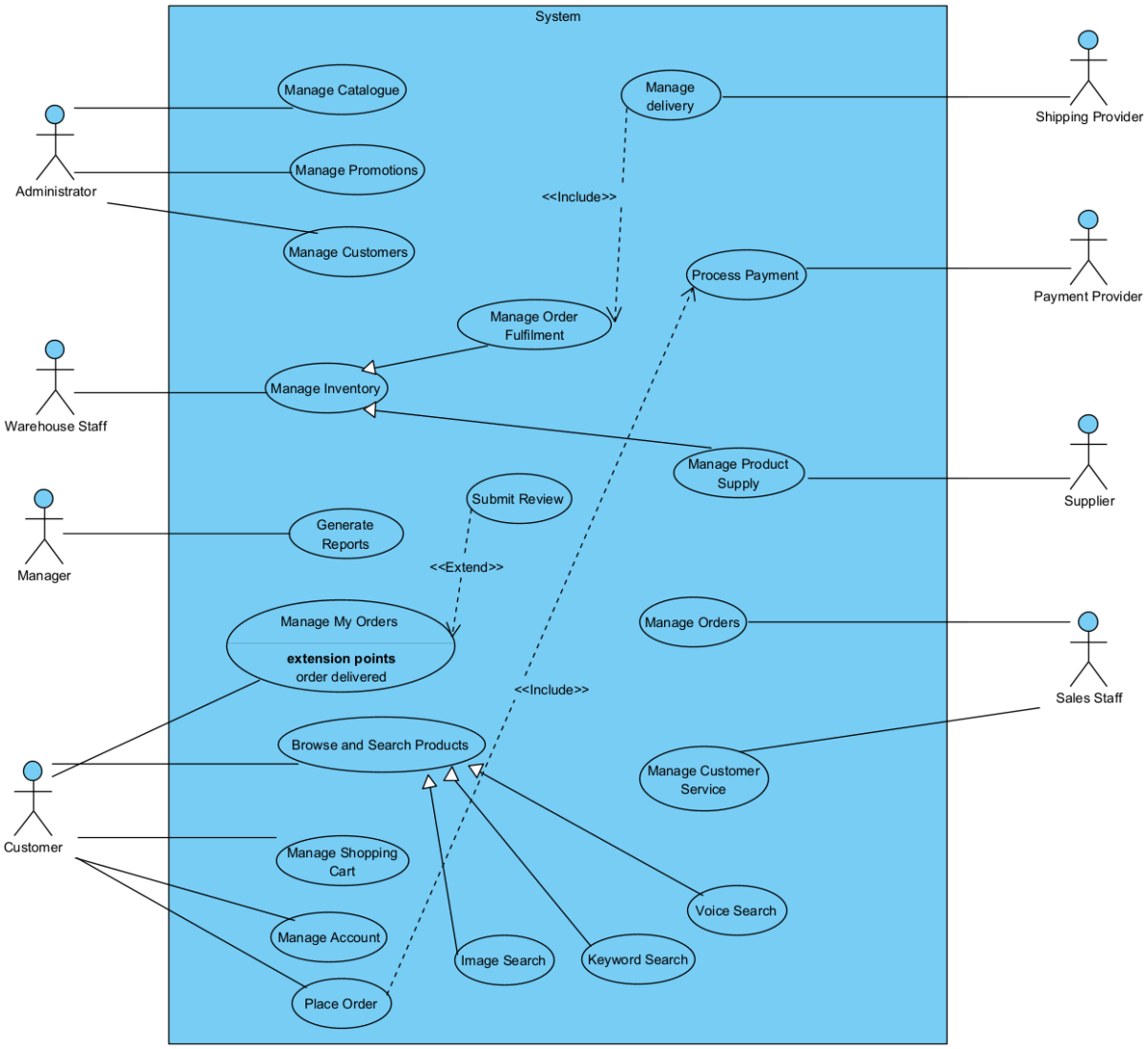


Figure 2. Order-and-payment activity diagram

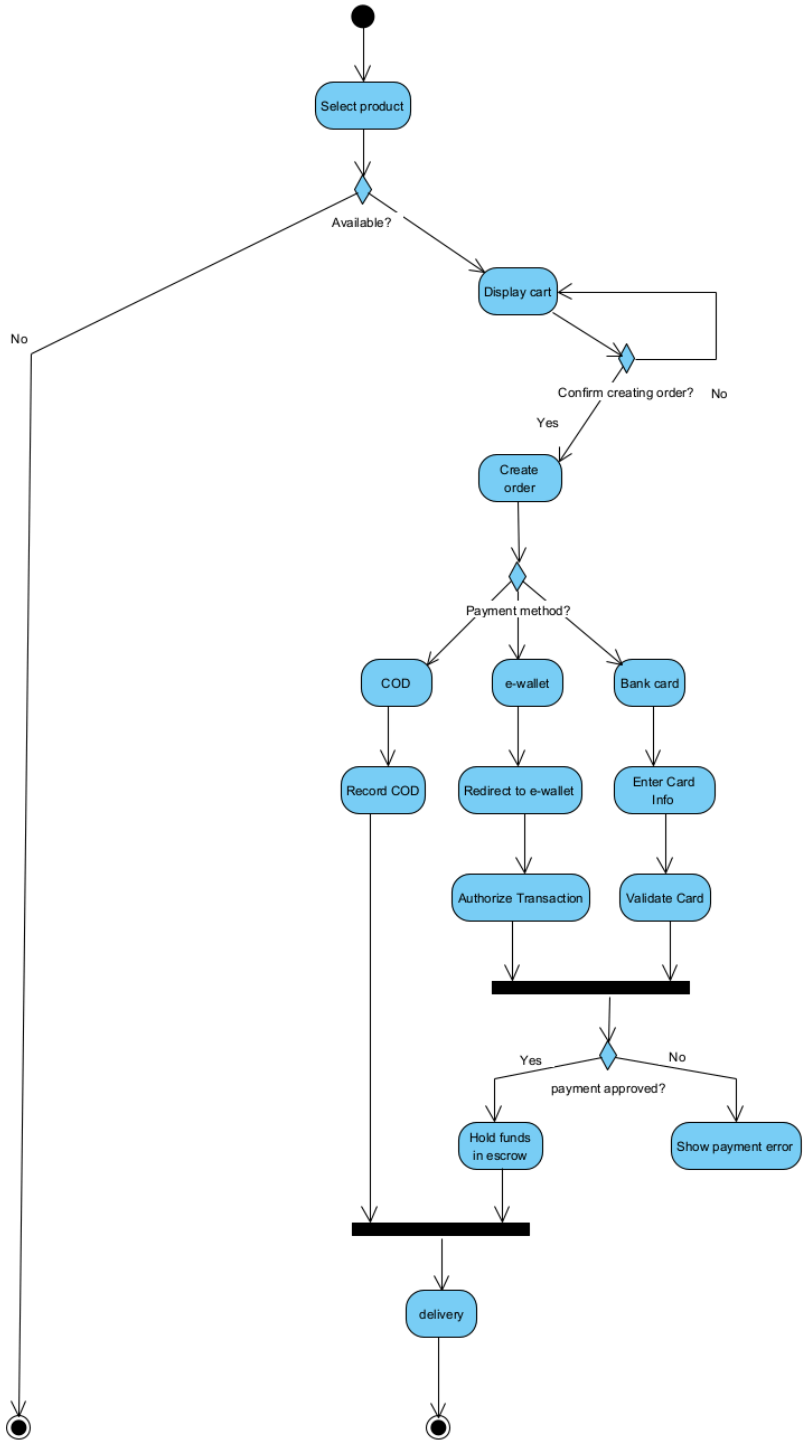


Figure 3. Domain entity class diagram

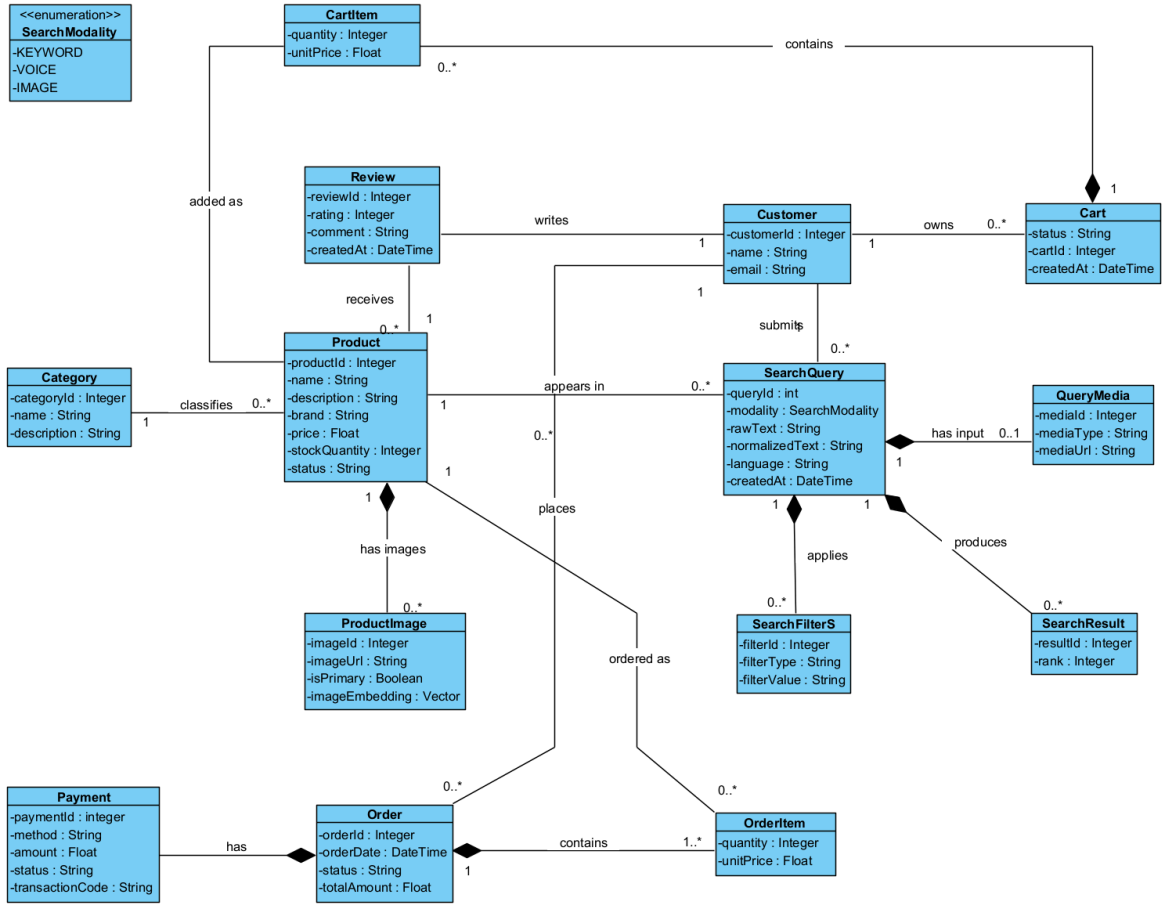


Figure 4. Analysis sequence diagram for a search request

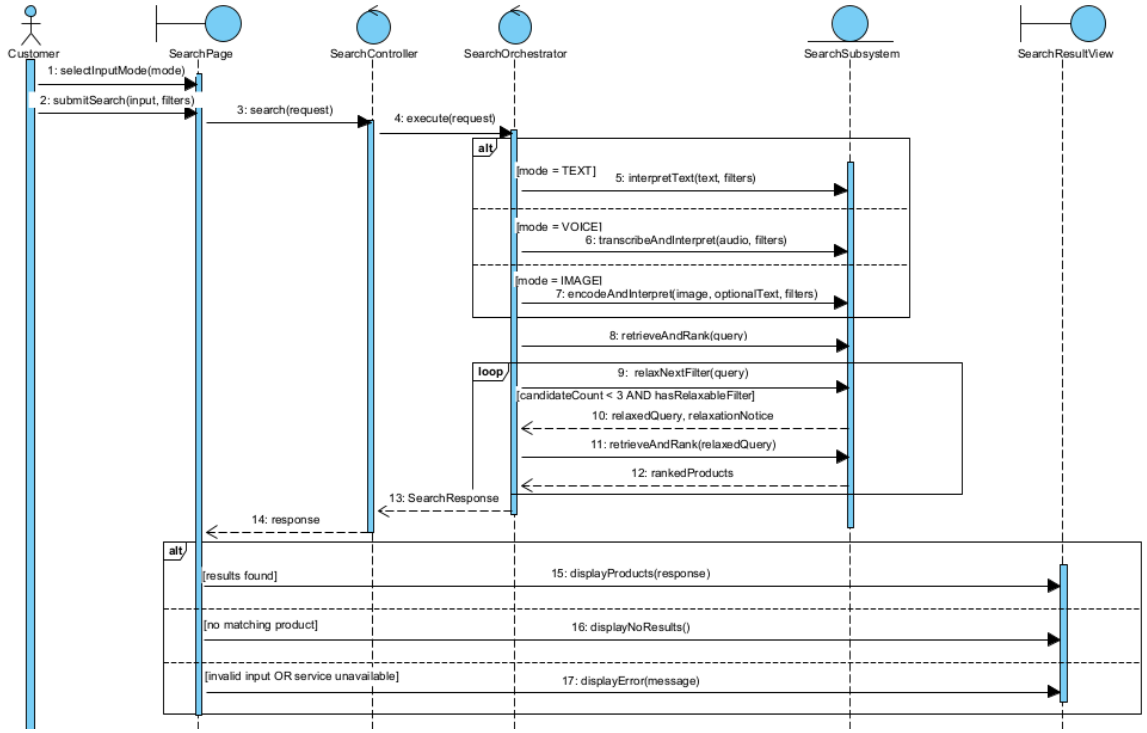


Figure 5. Three-layer architecture and external integrations

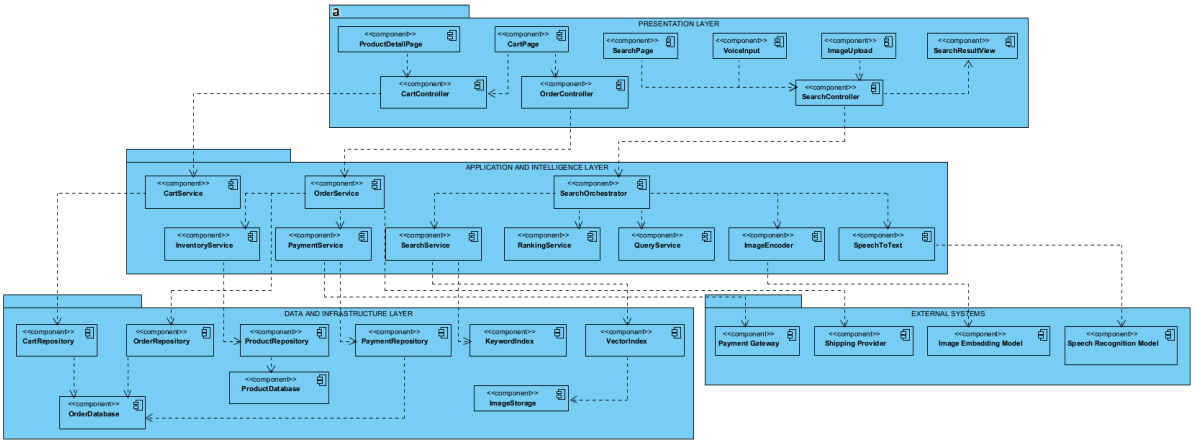


Figure 6. Design class diagram for multimodal search

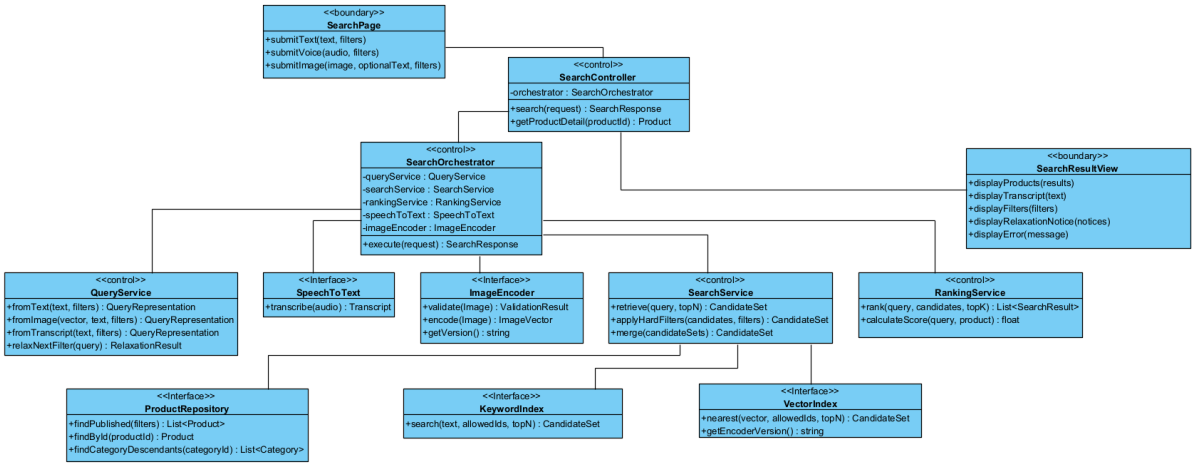


Figure 7. Physical data model for search and commerce

